



Facultatea de Automatică și Calculatoare

An I, grupa 30214

Sistem de iluminare variabilă

Profesor:
Timar Mihai Cristian

Student:
Toderașco Ion

Cuprins

1. Specificația proiectului	3
2. Fenomenul de PWM (pulse-width modulation)	4
3. Schema bloc a aplicației cu componentele principale	5
4. Unitatea de comandă și cea de execuție	5
5. Etapele de proiectare:	
5.1 Lista componentelor utilizate	6
5.2 Semnificația notațiilor efectuate și a pinilor interfeței cu exteriorul	13
5.3 Justificarea soluției alese	13
5.4 Instrucțiuni de utilizare și întreținere	14
5.5 Posibilități de dezvoltare ulterioară	17

Specificația proiectului

Să se proiecteze un sistem de iluminare variabilă cu leduri de pe plăcile cu FPGA. Se cer două moduri de funcționare:

1. Mod automat – ledurile pulsează la intervale de timp 2 secunde și 5 secunde
2. Mod manual – intensitatea luminii ledurilor se citește de pe întrerupătoare

Pentru varierea intensității luminoase se va folosi PWM (pulse-width modulation). Să se implimenteze proiectul pe o placă FPGA Nexys 2.

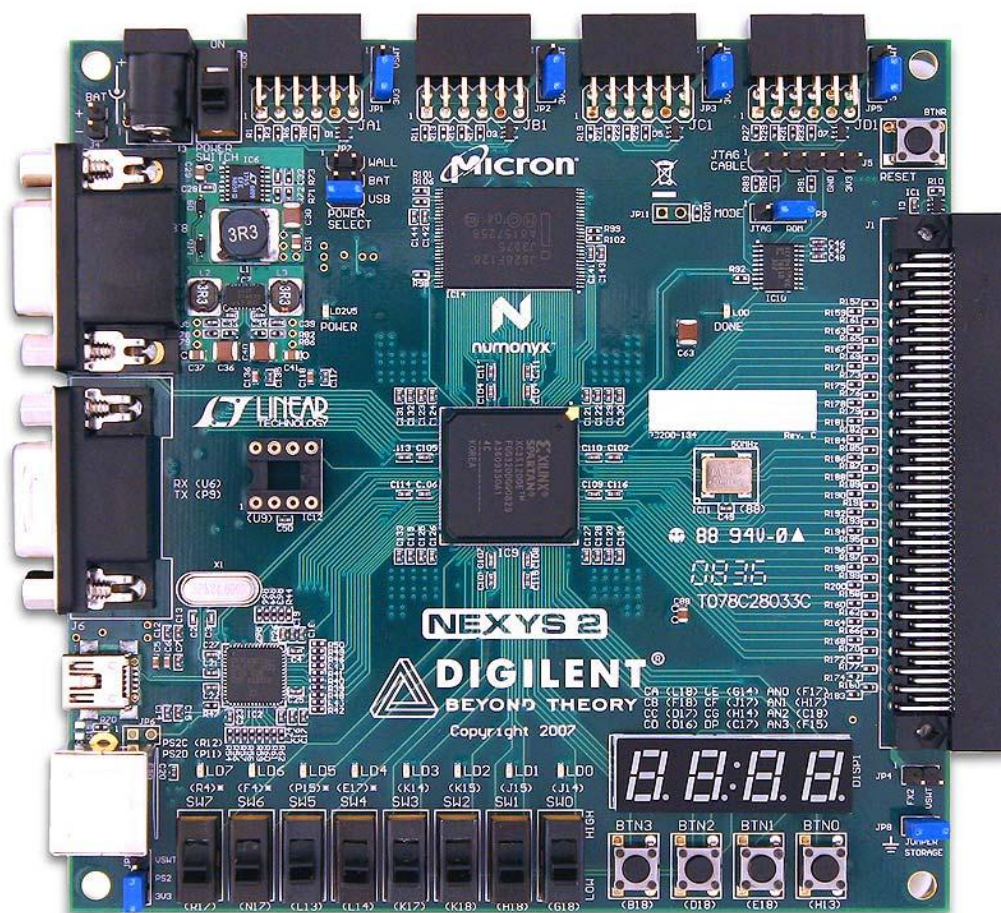


Fig. 1. Placa NEXYS 2

Pulse-width modulation

Pulse Width Modulation, sau PWM este o tehnică folosită pentru a varia în mod controlat tensiunea dată unui dispozitiv electronic. Această motodă schimbă foarte rapid tensiunea oferită dispozitivului respectiv din ON (1 logic) în OFF (0 logic) și invers. Perioada de timp corespunzătoare valorii ON dintr-un ciclu ON-OFF se numește factor de umplere (duty cycle) și reprezintă, în medie, ce tensiune va primi dispozitivul electronic. Astfel, se pot controla circuitele analogice din domeniul digital. Practic, asta înseamnă că un LED acționat astfel se va putea aprinde / stinge gradual.

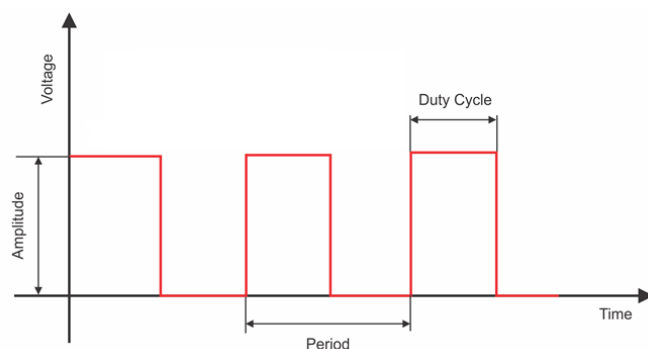


Fig.2. Forma de undă a unui semnal PWM

Factorul de umplere (Duty cycle) se exprimă în procente și reprezintă cât la sută din perioada unui semnal acesta va fi pe nivelul ON (1 logic). În figura 3 se pot observa semnalele PWM cu factori de umplere diferiți. Astfel, se poate deduce foarte ușor formula pentru a obține valoarea factorului de umplere (D): $D = \frac{\tau}{T}$

Unde: τ – este durata impulsului, iar T – este perioada semnalului

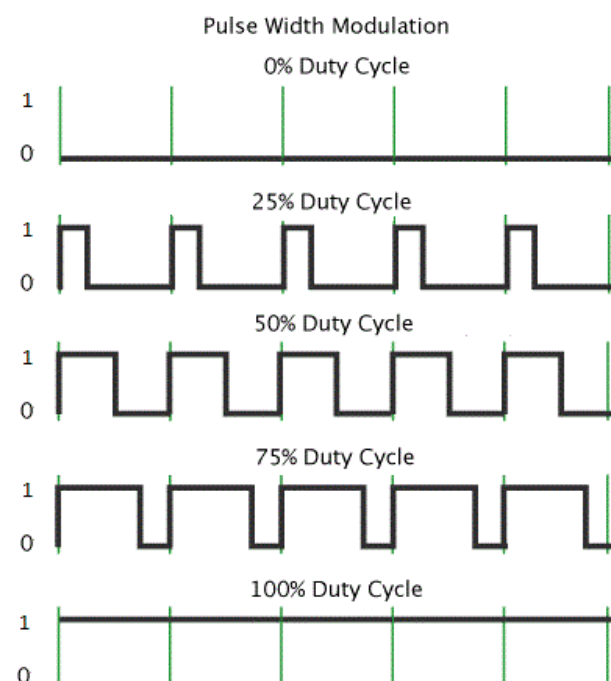


Fig.3. Exemplu de undă Duty Cycle

Schema bloc a aplicației

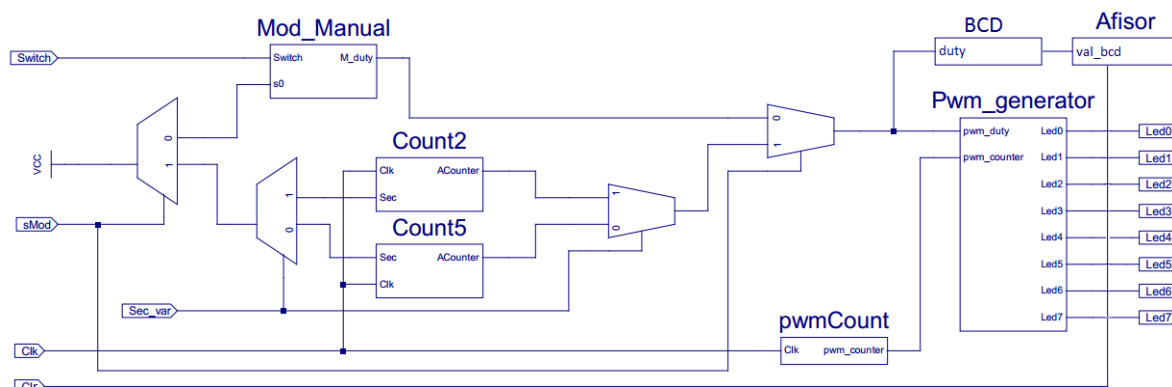


Fig.4. Schema bloc a aplicației cu evidențierea principalelor componente

Unitatea de comanda și cea de execuție

Unitatea de comandă diferă pentru cele două moduri (automat, manual). Pentru modul manual unitatea de comandă reprezintă întrerupătoarele care comandă valoarea Duty Cycle ca va fi recepționată de generatorul de undă PWM.

Pentru modul automat unitatea de comandă reprezintă semnalele interne **sec** și **sel** care dictează care din numărătoare să numere la momentul dat și pe care domeniu (crescător / descrescător).

Având un semnal de Duty Cycle generat în mod manual sau automat acesta este preluat de către generatorul de PWM care reprezintă unitatea de execuție. Generatorul de undă PWM preia valoarea DC și generează o undă PWM care impune led-urilor valoarea 1 logic sau 0 logic la un moment anumit.

La fel ca unitate de execuție poate fi privit decodificatorul BCD al semnalului de Duty Cycle ce generează un bit vector pentru afișorul pe 7 segmente.

Etapele de proiectare

Lista componentelor utilizate

1. Componenta Manuală

```
-- Valoarea DC incarcata de pe switch-uri
signal dutyM: STD_LOGIC_VECTOR (6 downto 0);
-- Valoarea dutyM in integer
signal M_duty: integer := 0;

.....

-- Componenta manuala
dutyM <= Switch;
-- Convert from bit_vector to integer
M_duty <= TO_INTEGER(unsigned(dutyM));
```

dutyM este un semnal care memoreaza valoarea introdusă pe switch-uri

M_duty este semnalul generat de componenta manuală

2. Componenta Automată

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;
use IEEE.NUMERIC_STD.all;

entity pwm_automat is
    port(clk, sMod: in STD_LOGIC;
          aduty: out STD_LOGIC_VECTOR(7 downto 0));
end entity;

architecture pwm_automat of pwm_automat is
    -- Numarator componenta automata; Valoarea DC generat
    signal Acounter, A_duty: integer := 0;
    -- Selectie Count-up / Count-down; Selectie Numarator 2s / 5s
    signal sel, sec: STD_LOGIC := '1';
begin
    -- Componenta automata
    process(clk, sec, sMod)
    begin
        if sMod = '1' then      -- Enable Counters
```

```

if sec = '1' then -- Counter clock 2s
    if clk'event and clk = '1' then
        Acounter <= Acounter + 1;
        if Acounter = 390624 and sel = '1' then
            -- Count-up numarator 2s
            Acounter <= 0;
            A_duty <= A_duty + 1;

            if A_duty = 128 then
                A_duty <= A_duty - 1;
                sel <= '0';
            end if;
        elsif Acounter = 390624 and sel = '0' then
            -- Count-down numarator 2s
            Acounter <= 0;
            A_duty <= A_duty - 1;

            if A_duty = 1 then
                sec <= '0';
                sel <= '1';
            end if;
        end if;
    end if; -- End Counter clock 2s

elsif sec = '0' then -- Counter 5s
    if clk'event and clk = '1' then
        Acounter <= Acounter + 1;

        if Acounter = 976562 and sel = '1' then
            -- Count-up numarator 5s
            Acounter <= 0;
            A_duty <= A_duty + 1;

            if A_duty = 128 then
                A_duty <= A_duty - 1;
                sel <= '0';
            end if;
        elsif Acounter = 976562 and sel = '0' then
            -- Count-down numarator 5s
            Acounter <= 0;
            A_duty <= A_duty - 1;

            if A_duty = 1 then
                sec <= '1';
                sel <= '1';
            end if;
        end if;
    end if; -- End Counter 5s
end if; -- End Counters
end if; -- End enable counters
end process;
-- End componenta Automata

aduty <= std_logic_vector(to_unsigned(A_duty, 8));

end pwm_automat;

```

Componenta automata constă din două numărătoare reversibile de la 0 la 127. Limita număratorului este dictată de numărul de switch-uri disponibile pentru componenta manuală, cu 7 switch-uri putem reprezenta cea mai mare valoare "111 1111" ce înseamnă 127. Conform condiției LED-ul trebuie să pulseze automat în intervale de timp de 2 secunde și 5 secunde. Pentru a realiza această cerință primul numărător numără de la 0 la 127 în 1 secundă și de la 127 la 0 în 1 secundă, celălalt numărător respectiv în 2,5 , 2,5 secunde.

Utilizând clock-ul implicit al plăcii FPGA Nexys 2, care are o frecvență de 50MHz, s-au efectuat următoarele calcule astfel încât numărătoarele să numere în timpii necesari.

$$2s = 2\,000\,000\,000\,000\,ns$$

$$5s = 5\,000\,000\,000\,000\,ns$$

Se efectuează următoarele operații pentru a determina peste câte nanosecunde numărătorul poate incrementa valoarea semnalului de DC (Duty Cycle) generat.

$$\frac{2\,000\,000\,000\,ns}{20\,ns} = 100\,000\,000$$

Se realizează împărțirea timpului necesar la 20ns care reprezintă frecvența clock-ului implicit.

Următoarea operație este împărțirea numărului obținut la 2, deoarece avem 2 stări de numărare: crescător și descrescător.

$$\frac{100\,000\,000}{2} = 50\,000\,000$$

Ultima operație este împărțirea la 128, care reprezintă limita de numărare.

$$\frac{50\,000\,000}{128} = 390625$$

Respectiv pentru 5s efectuăm aceleași operații și obținem valoarea de 976563.

În cod la descrierea numărătoarelor folosim aceste valori scăzând o unitate deoarece numărătorul numără de la 0.

La final valoarea generată de numărător e convertită într-un bit vector pentru a putea fi folosită în programul principal.

3. Componenta PWM

```
-- PWM counter
process(clk)
begin
    if clk'event and clk = '1' then
        pwm_counter <= pwm_counter + 1;
        if pwm_counter = 127 then
            pwm_counter <= 0;
        end if;
    end if;
end process;

-- PWM DC
process(sMod, M_duty, A_duty)
begin
    if sMod = '0' then
        pwm_duty <= M_duty;
    else
        pwm_duty <= A_duty;
    end if;
end process;

-- PWM signal
process(pwm_counter, pwm_duty)
begin
    if pwm_duty = 0 then
        pwm_led <= (others => '0');
    elsif pwm_counter < pwm_duty then
        pwm_led <= (others => '1');
    else
        pwm_led <= (others => '0');
    end if;
end process;
```

Componenta PWM este alcătuită dintr-un multiplexor care în dependență de selecție atribuie semnalului **pwm_duty** valoarea DC necesară (generată manual sau automat). Un numărător al segmentelor de perioadă al semnalului pwm și însăși generatorul de semnal pwm.

4. Decodificatorul BCD

Decodificatorul BCD este folosit pentru a prelucra valoarea curentă a semnalului DC generând un bit vector pentru componenta afișorului pe 7 segmente.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_SIGNED.all;

entity bcd is
    port(in_vect: in STD_LOGIC_VECTOR(7 downto 0);
         out_vect: out STD_LOGIC_VECTOR(15 downto 0));
end bcd;

architecture pwm_bcd of bcd is

    function conv_bcd(bin: STD_LOGIC_VECTOR(7 downto 0)) return
    STD_LOGIC_VECTOR is
        variable i: integer := 0;
        variable bcd: STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
        variable bint: STD_LOGIC_VECTOR(7 downto 0) := bin;
    begin
        for i in 0 to 7 loop
            bcd(15 downto 1) := bcd(14 downto 0);
            bcd(0) := bint(7);
            bint(7 downto 1) := bint(6 downto 0);
            bint(0) := '0';

            if(i < 7 and bcd(3 downto 0) > "0100") then
                bcd(3 downto 0) := bcd(3 downto 0) + "0011";
            end if;

            if(i < 7 and bcd(7 downto 4) > "0100") then
                bcd(7 downto 4) := bcd(7 downto 4) + "0011";
            end if;

            if(i < 7 and bcd(11 downto 8) > "0100") then
                bcd(11 downto 8) := bcd(11 downto 8) + "0011";
            end if;

            if(i < 7 and bcd(15 downto 12) > "0100") then
                bcd(15 downto 12) := bcd(15 downto 12) + "0011";
            end if;
        end loop;

        return bcd;
    end conv_bcd;

begin
    process(in_vect)
    begin
        out_vect <= conv_bcd(in_vect);
    end process;
end pwm_bcd;

```

Decodificatorul constă din funcția **conv_bcd** care transformă un bit vector pe 8 biți într-un vector de 16 biți, câte 4 biți pentru fiecare afișor, folosind algoritmul *"double dabble"*. La final procesăm vectorul de intrare prin funcția noastră și obținem un bit vector decodificat pe 16 biți.

Principiul de funcționare al algoritmului "*double dabble*" este următorul. Se ia un bit vector de 16 biți inițializat cu zero și valoarea în binar (pe 8 biți) a numărului pe care dorim să-l decodificăm. Se șiftează câte un bit și în cazul în care avem un număr mai mare decât 4 îl încrementăm cu 3. Mai jos este demonstrat cum lucrează funcția pe exemplul numărului 243_{10} .

0000	0000	0000	1111	0011	Inițializare 243_2
0000	0000	0001	1110	0110	Șiftare 1 bit
0000	0000	0011	1100	1100	Șiftare 1 bit
0000	0000	0111	1001	1000	Șiftare 1 bit
0000	0000	1010	1001	1000	Adunăm 3 la unități, valoarea trecută - 7
0000	0001	0101	0011	0000	Șiftare 1 bit
0000	0001	1000	0011	0000	Adunăm 3 la unități, valoarea trecută - 5
0000	0011	0000	0110	0000	Șiftare 1 bit
0000	0110	0000	1100	0000	Șiftare 1 bit
0000	1001	0000	1100	0000	Adunăm 3 la zeci, valoarea trecută - 6
0001	0010	0001	1000	0000	Șiftare 1 bit
0010	0100	0011	0000	0000	Șiftare 1 bit
BCD	2	4	3		

5. Afișorul pe 7 segmente

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

entity afis is
    port(value: in STD_LOGIC_VECTOR(15 downto 0);
          clk: in STD_LOGIC;
          clr: in STD_LOGIC;
          display: out STD_LOGIC_VECTOR(6 downto 0);
          anod: out STD_LOGIC_VECTOR(3 downto 0));
end entity;

architecture pwm_afisor of afis is
    signal sel: STD_LOGIC_VECTOR(1 downto 0); -- selectie
    signal digit: STD_LOGIC_VECTOR(3 downto 0); -- valoarea afisorului
    signal clk_div: STD_LOGIC_VECTOR(18 downto 0); -- divizor de clock
begin
    sel <= clk_div(18 downto 17); -- Selectia

    process(sel, value)
    begin
        case sel is
            when "00" => digit <= value(3 downto 0);
            when "01" => digit <= value(7 downto 4);
            when "10" => digit <= value(11 downto 8);
        end case;
    end process;
end architecture;

```

```

        when others => digit <= value(15 downto 12);
    end case;
end process;

process(digit)
begin
    case digit is
        when "0000" => display <= "1000000"; -- 0
        when "0001" => display <= "1111001"; -- 1
        when "0010" => display <= "0100100"; -- 2
        when "0011" => display <= "0110000"; -- 3
        when "0100" => display <= "0011001"; -- 4
        when "0101" => display <= "0010010"; -- 5
        when "0110" => display <= "0000010"; -- 6
        when "0111" => display <= "1111000"; -- 7
        when "1000" => display <= "0000000"; -- 8
        when "1001" => display <= "0010000"; -- 9
        when others => display <= "1111111";
    end case;
end process;

process(sel, clr)
begin
    anod <= "1111";
    if clr = '0' then
        anod(conv_integer(sel)) <= '0';
    end if;
end process;

process(clk, clr)
begin
    if clr = '1' then
        clk_div <= (others => '0');
    elsif clk'event and clk = '1' then
        clk_div <= clk_div + 1;
    end if;
end process;

end pwm_afisor;

```

Lista componentelor utilizate:

- Porți logice
- Numărătoare
- Multiplexor
- Demultiplexor
- Decodificator
- Comparator

Semnificația notațiilor efectuate și a pinilor interfeței cu exteriorul

Încărcarea manuală a DC (Duty Cycle) pentru componenta manuală se efectuează pe întrerupătoare, astfel:

Switch (SW6 – SW0) un șir de biți. Valoarea implicită (pentru toate întrerupătoarele deschise) "000 0000" care reprezintă 0% (LED stins).

Intrerupătorul 0 reprezintă ultimul bit (pentru "000 0001" valoarea PWM va fi 1), întrerupătorul 6 cel mai semnificativ bit (pentru "100 0000" valoarea PWM va fi 64).

sMOD (SW7) selecție între modul manual și cel automat. În modul implicit când întrerupătorul este deschis selecția este făcută pentru componenta manuală. Pentru a porni componenta automată este suficient să închidem întrerupătorul.

btn_clr (BTN0) semnalul de "clear" pentru afișoarele pe 7 segmente.

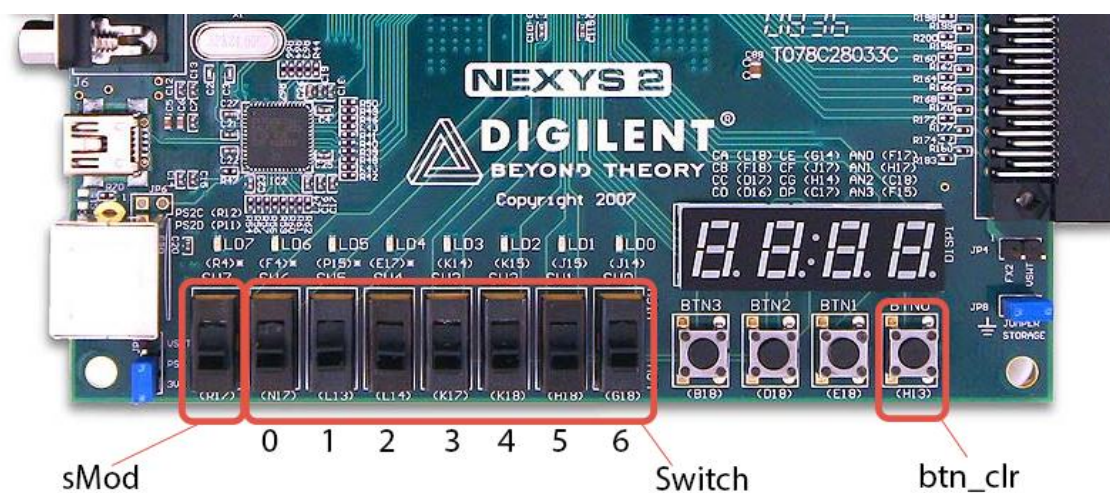


Fig.5. Semnificația pinilor interfeței cu exteriorul

Justificarea soluției alese

La realizarea acestui proiect am optat pentru obținerea unui cod lizibil și ușor de înțeles de către o altă persoană. Pentru aceasta am utilizat nume semnificative pentru intrări și ieșiri, am scris comentarii pentru a evidenția segmentele de cod și pentru a descrie semnificația unei linii de cod anumite.

Am evitat obținerea unui cod sursă foarte mare, din această cauză am separat secvențele de cod pe componente.

Pentru generatorul de semnal PWM am utilizat componente simple, numărător, multiplexor, comparator.

Ușurința în utilizare e datorată puținelor comenzi care depind de utilizator și de afișoarele pe 7 segmente care ajută utilizatorul să înțeleagă fenomenul de PWM.

Toate în ansamblu reprezintă o soluție eficientă pentru proiectul respectiv.

Instrucțiuni de utilizare și întreținere

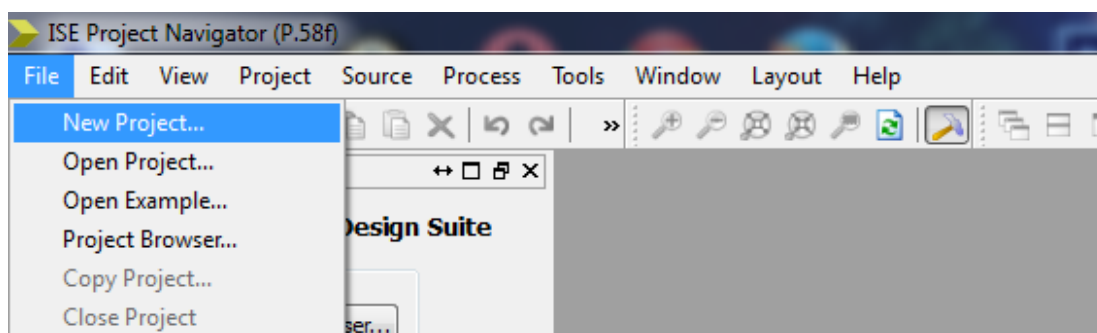
Proiectul dat presupune utilizarea plăcii FPGA Nexys 2 și respectiv utilitarul soft *Xilinx ISE Design Suite*.

Pentru început e necesară sintetizarea codului, asignarea pinilor și generarea fișierului cu extensia **"*.bit"**, după care e posibilă încărcarea proiectului pe placa FPGA.

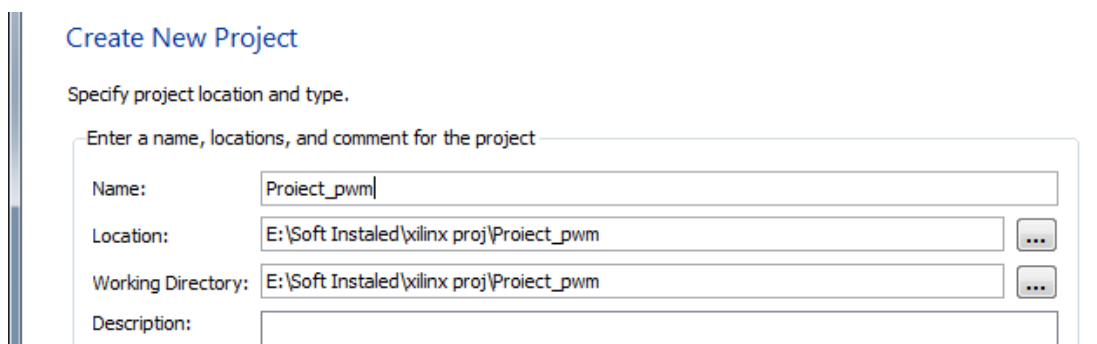
Pentru aceasta realizăm următorii pași:

1. Lansăm aplicația *Xilinx ISE Design Suite* și creăm un proiect nou:

File > New Project



2. Se asociază un nume

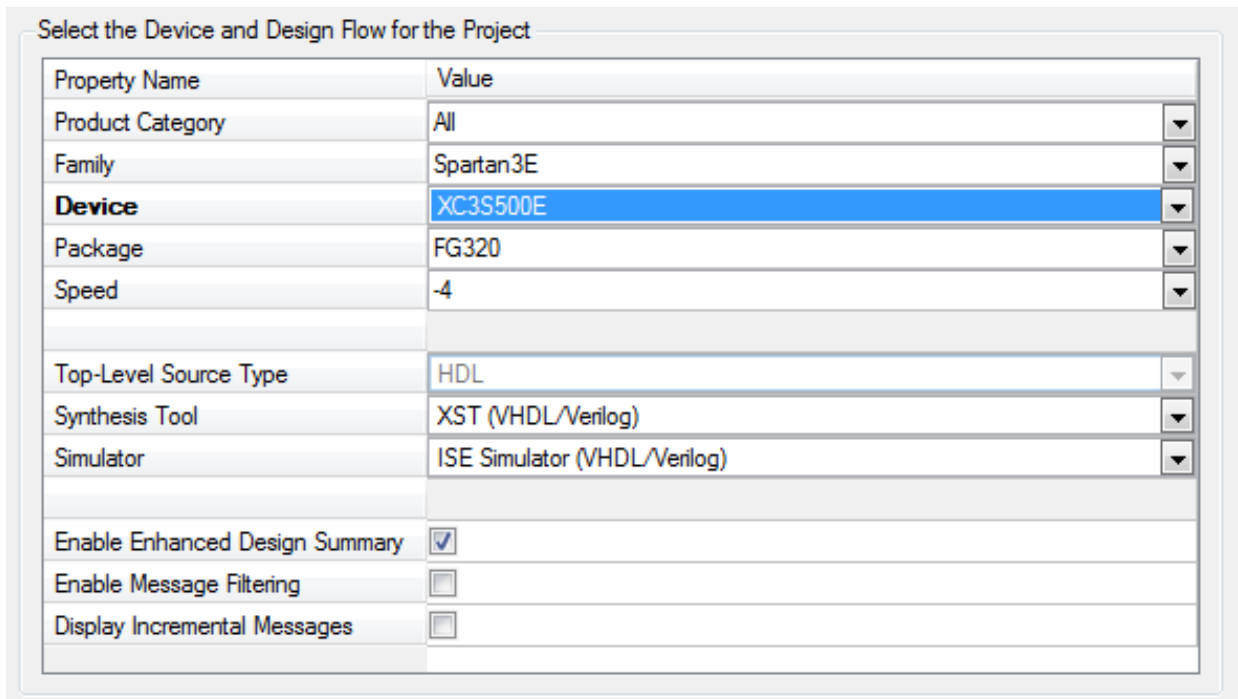


3. Se configurează setările corespunzătoare pentru placa Nexys 2 după cum urmează:

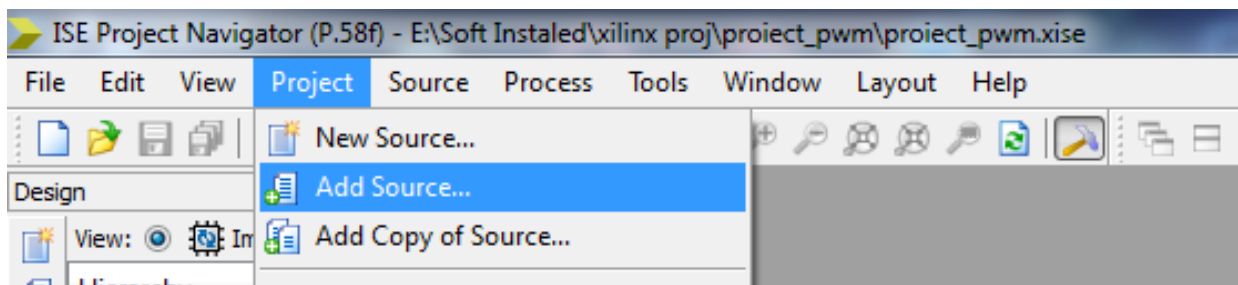
Family:	SPARTAN 3E
Device:	XC3S500E
Package:	FG320
Synthesis Tool:	XST(VHDL/Verilog)
Simulator:	ISE Simulator (VHDL/Verilog)

Preferred Language: VHDL

Speed: -4

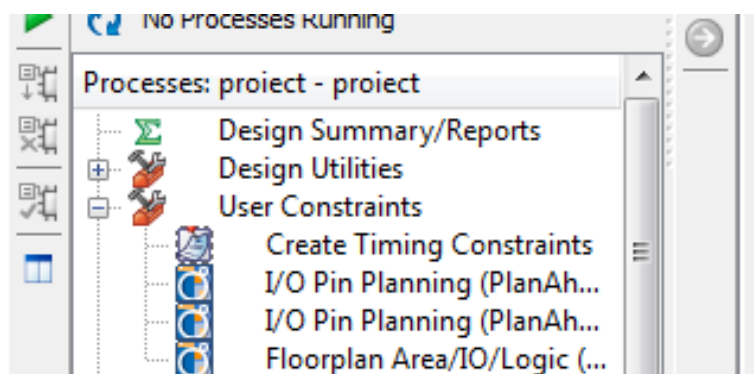


4. Se adaugă fișierul-sursă: **Project > Add Source ...**



Se găsește sursa fișierului *.**vhd** și se adaugă în program. Se adaugă la fel și celelalte componentele necesare.

5. Se asignează pinii pentru intrările și ieșirile proiectului



Dublu click pe **Create Timing Constraints**. Se va genera un fișier *.**ucf** de care avem nevoie.

Deschidem fișierul respectiv, introducem următorul cod și salvăm fișierul (Ctrl+s).

```
NET "Clk" LOC=B8;
NET "sMod" LOC=R17;
NET "btn_clr" LOC=B18;

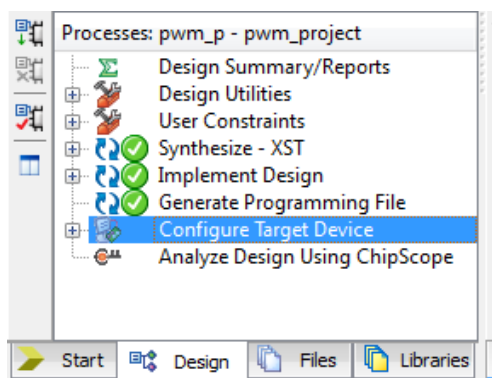
NET "Switch(0)" LOC=N17;
NET "Switch(1)" LOC=L13;
NET "Switch(2)" LOC=L14;
NET "Switch(3)" LOC=K17;
NET "Switch(4)" LOC=K18;
NET "Switch(5)" LOC=H18;
NET "Switch(6)" LOC=G18;

NET "pwm_led(0)" LOC=P4;
NET "pwm_led(1)" LOC=E4;
NET "pwm_led(2)" LOC=P16;
NET "pwm_led(3)" LOC=E16;
NET "pwm_led(4)" LOC=K14;
NET "pwm_led(5)" LOC=K15;
NET "pwm_led(6)" LOC=J15;
NET "pwm_led(7)" LOC=J14;

NET "a_segment(0)" LOC=L18;
NET "a_segment(1)" LOC=F18;
NET "a_segment(2)" LOC=D17;
NET "a_segment(3)" LOC=D16;
NET "a_segment(4)" LOC=G14;
NET "a_segment(5)" LOC=J17;
NET "a_segment(6)" LOC=H14;

NET "a_anod(0)" LOC=F17;
NET "a_anod(1)" LOC=H17;
NET "a_anod(2)" LOC=C18;
NET "a_anod(3)" LOC=F15;
```

6. Verificăm proiectul și generăm fișierul de execuție



Facem dublu click pe: *Synthesize – XST, Implement Design, Generate Programming File*. Când în stânga acestor trei procese va fi o bifă alba pe fundal verde (ca în imagine) putem trece la următorul pas.

Configure Target Device – se lansează programul **Adept** (utilitarul folosit pentru programarea plăcii), se conectează placa FPGA Nexys 2 prin cablu USB la calculator, se accesează comanda **Initialize Chain**, din utilitarul Adept, se introduce fișierul cu extensia **".bit"** generat de Xilinx, apoi se accesează **Program**.

Posibilități de dezvoltare ulterioară

Proiectul dat poate fi dezvoltat prin adăugarea de noi funcționalități ca:

1. Dezvoltarea componentei manuale prin adăugarea a două butoane: Count up și Count down, care vor crește, respectiv descrește valoarea DC manuală.
2. Crearea de varietăți pentru componenta automată.
3. Șiftarea pe led-uri, DC cel mai mare să fie urmat de DC mai mici, valori precedente.